****Experiment 7: Random Forest Classifier on Nonlinear Data**

Aim / Objective

Write a program in python to implement Random Forest Classifier on Nonlinear Data.

Theory / Introduction

Random Forest is an ensemble learning method that combines multiple decision trees to improve classification accuracy and control overfitting.

- Each tree is trained on a different bootstrap sample of the data (bagging).
- Final prediction is based on the majority vote of individual trees.
- It works well with nonlinear and high-dimensional data.

Random Forest handles nonlinear patterns better than linear models, especially when decision boundaries are complex.

Software / Tools Required

- Python
- Scikit-learn
- Matplotlib
- NumPy

Dataset Used

Moons Dataset (Synthetic)

About the Dataset

The **Moons dataset** is a toy binary classification dataset with two interleaving half-circle shapes. It is often used to test classifiers on nonlinear decision boundaries.

- 2 input features (X1, X2)
- 1 target label (0 or 1)

🔗 Available from Scikit-learn:

from sklearn.datasets import make_moons

Procedure / Steps

1. Generate synthetic data using make_moons()
2. Split the dataset into training and testing sets
3. Train a Random Forest Classifier
4. Visualize the decision boundary
5. Evaluate model accuracy

Python Step-by-Step Code with Explanations

```
# Step 1: Import libraries
from sklearn.datasets import make_moons
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt
import numpy as np

# Step 2: Generate nonlinear data
X, y = make_moons(n_samples=300, noise=0.25, random_state=42)

# Step 3: Split the data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Step 4: Train the Random Forest model
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# Step 5: Predict and evaluate
y_pred = model.predict(X_test)
print("Test Accuracy:", accuracy_score(y_test, y_pred))
```

```
# Step 6: Plot decision boundary
h = 0.02
x_min, x_max = X[:, 0].min()-1, X[:, 0].max()+1
y_min, y_max = X[:, 1].min()-1, X[:, 1].max()+1
xx, yy = np.meshgrid(np.arange(x_min, x_max, h),
                     np.arange(y_min, y_max, h))
Z = model.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

plt.contourf(xx, yy, Z, alpha=0.4, cmap='coolwarm')
plt.scatter(X[:, 0], X[:, 1], c=y, edgecolors='k', cmap='coolwarm')
plt.title("Random Forest - Moons Data")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.grid(True)
plt.show()
```

Result / Output

- The Random Forest classifier achieved high accuracy on the test data.
- The decision boundary clearly separates the nonlinear clusters in the moons dataset.
- Visualization confirms model's strength in handling nonlinear separations.

Conclusion

The Random Forest algorithm proved effective for classifying complex, nonlinear patterns. Ensemble methods like Random Forest improve prediction accuracy and robustness by aggregating multiple models.

Viva Questions (2–3 per experiment)

1. What is the difference between bagging and boosting?
2. Why is Random Forest less prone to overfitting than a single decision tree?
3. How does Random Forest determine feature importance?